

DDD intro

LG Architecture Training Program

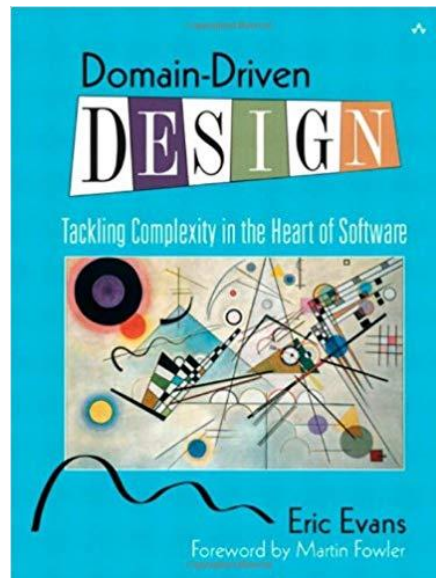
Paulo Merson



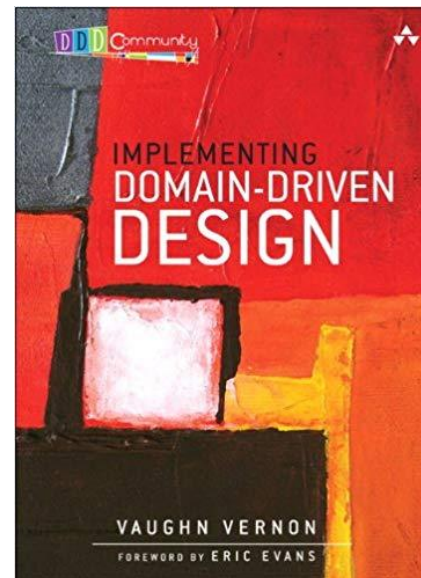
Domain-Driven Design (DDD)



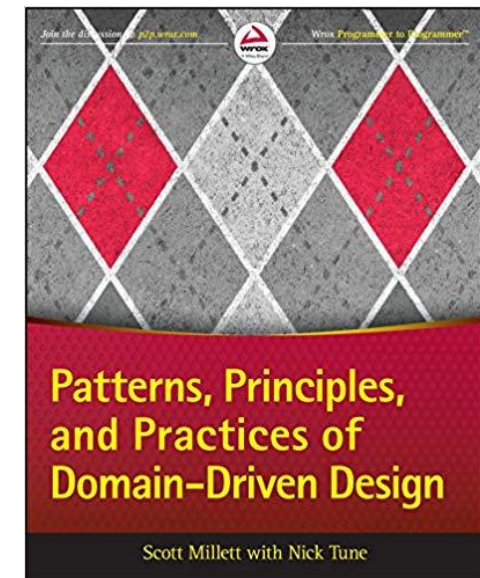
- DDD is an approach to domain modeling created by Eric Evans
- Good fit for modeling complex domain software systems
- A core tenet is collaboration between developers and domain experts



2003



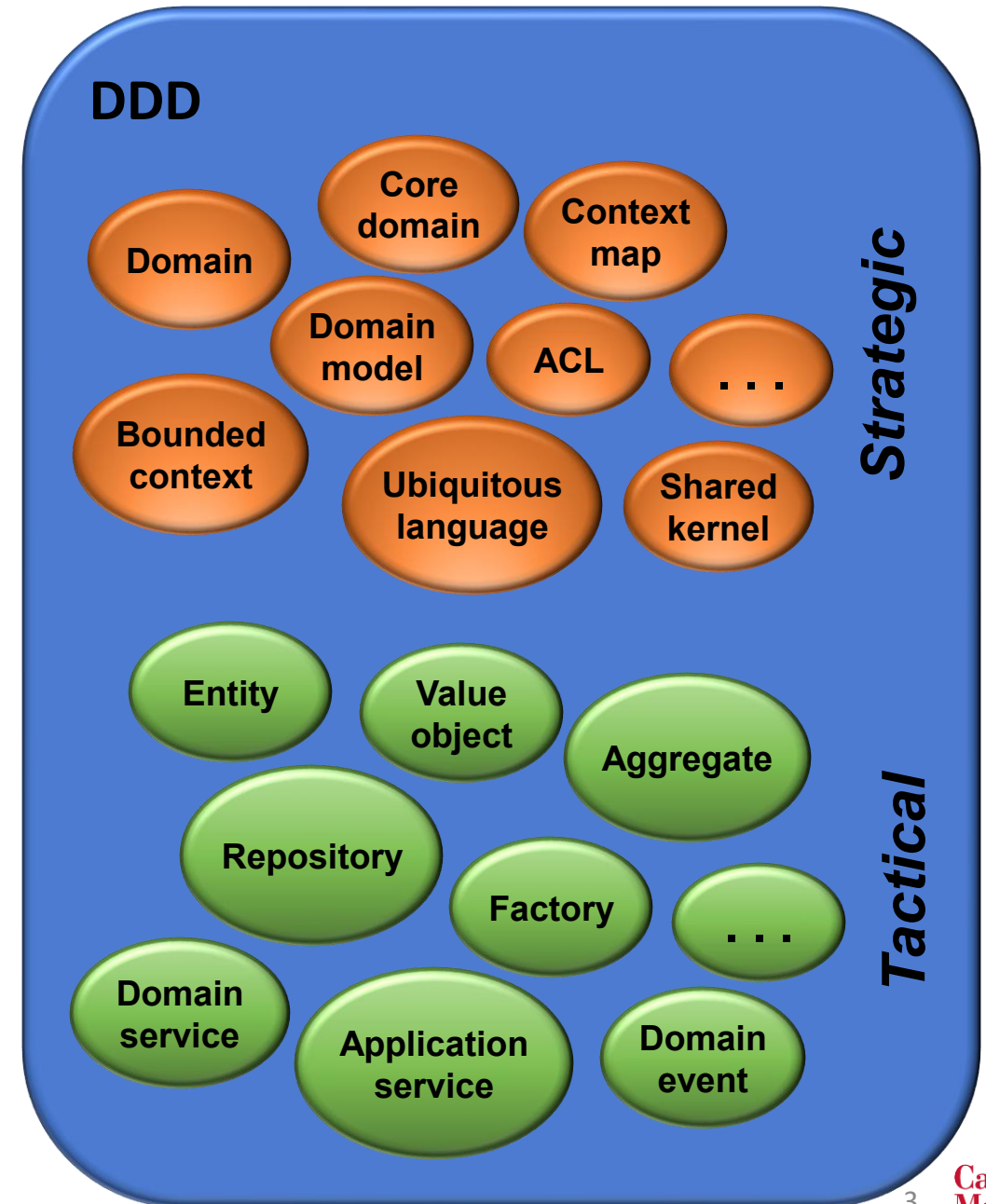
2013



2015

DDD main concepts

- Domain
- Aggregate
 - Entity, value object, aggregate root
- Bounded context
 - Context map, Anticorruption Layer
- Ubiquitous language
- Domain event
- Application service, domain service
- Repository



Domain

- Domain is the problem to be solved with software in an organization
- It includes the concepts and business rules needed to achieve the business goals of the organization
- Examples of organizations and their domains:
 - DHL: shipping parcels
 - Supreme Court: judicial cases involving the Constitution or federal law
 - Angelo's Pizza: produce and sell pizza

A domain is typically composed of subdomains

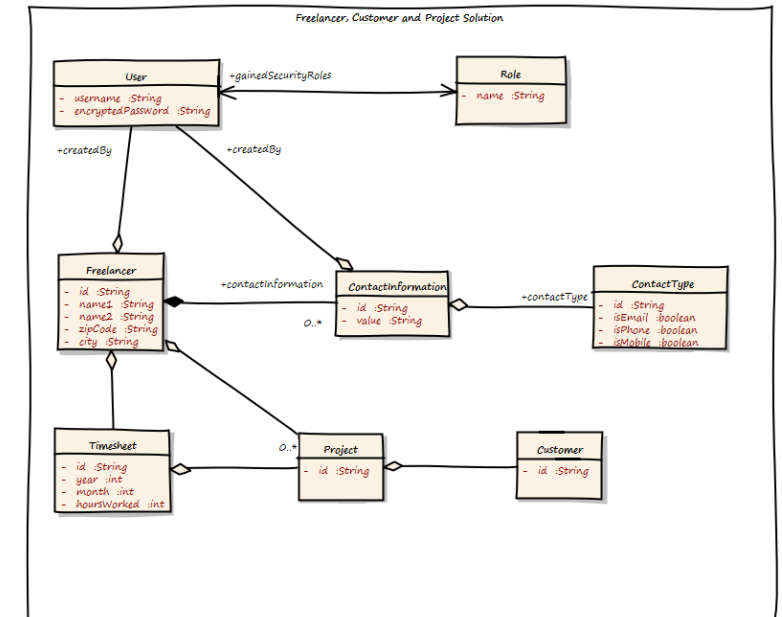
Core domain

- *Domain* is the generic term for a top-level domain or for subdomains
- A domain can be a
 - **core domain**—is crucial for the success of the organization
 - **supporting subdomain**—models important aspects of the business that are not core to the business
 - **generic subdomain**—required by the business in an auxiliary fashion

*The classification terms are not important;
identifying core domains is important*

Domain and Domain Model

- Domains are conceptual (problem space)
- Domain models describe the logical implementation (solution space)
- The scope of a domain model is determined by the bounded context [more on BC later]





*Each domain and subdomain
has its domain model*

~~enterprise model~~
~~shared database schema~~
~~unified field theory~~
~~one ring~~

There are always
multiple models.

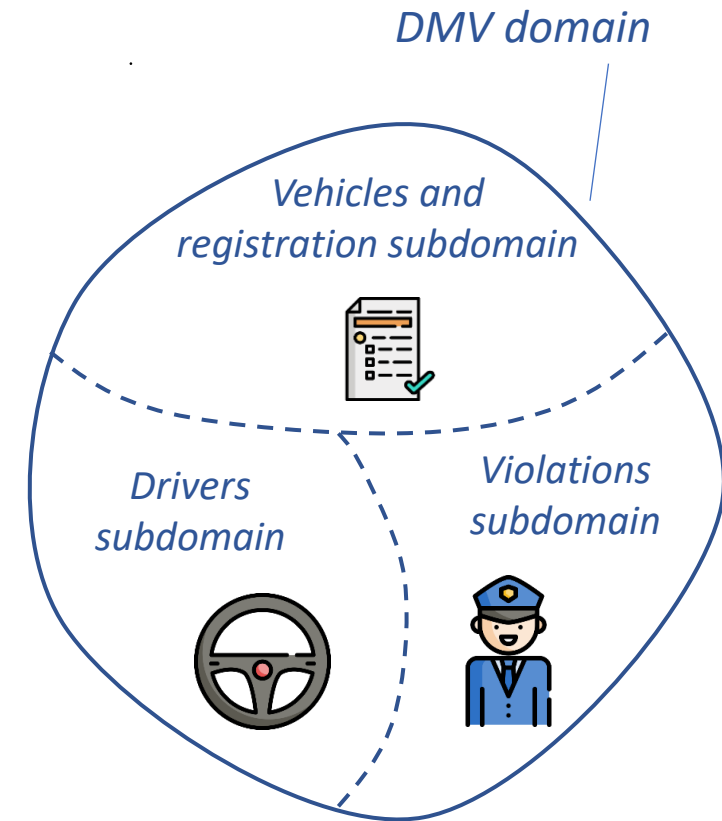


Filmed at
QCon London

Our running example – Springfield DOT/DMV

- Key functionality:
 - Issue a new driver's license
 - Renew a driver's license
 - Register a new vehicle
 - Transfer the title of a vehicle
 - Traffic ticket life cycle (issued, confirmed*, paid, cancelled, etc.)

* A confirmed ticket assigns points to a driver



Who
participates in
the modeling?

What's the
typical scope of
a model?

How do you
document a
domain model?

How do you
create a
domain model?

In particular, how
do you record
domain invariants?



Entity

- Entities have an ID and a life cycle
- Examples: Customer, Order, Payment
- Focus is on behavior, not data (rich object model)
 - The entity interface favors behaviors, not get/set functions

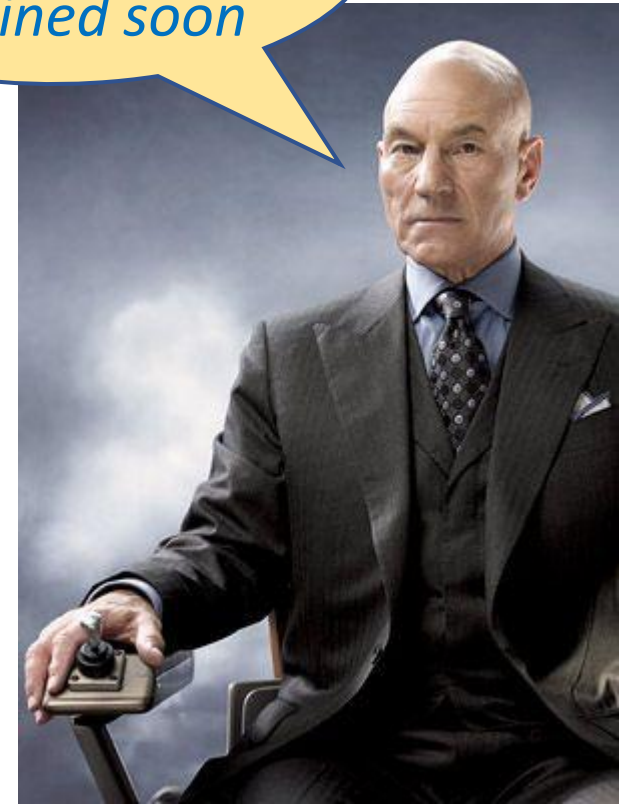
Avoid anemic domain models



Entities are rich and independent

- An entity should be self-validating
- Entities may have dependencies to
 - other entities in the same aggregate
 - value objects
 - domain events
 - domain services (for example, to send events)
- Entities should not have dependencies to
 - infrastructure elements
 - application services
 - repositories

These other DDD elements will be explained soon



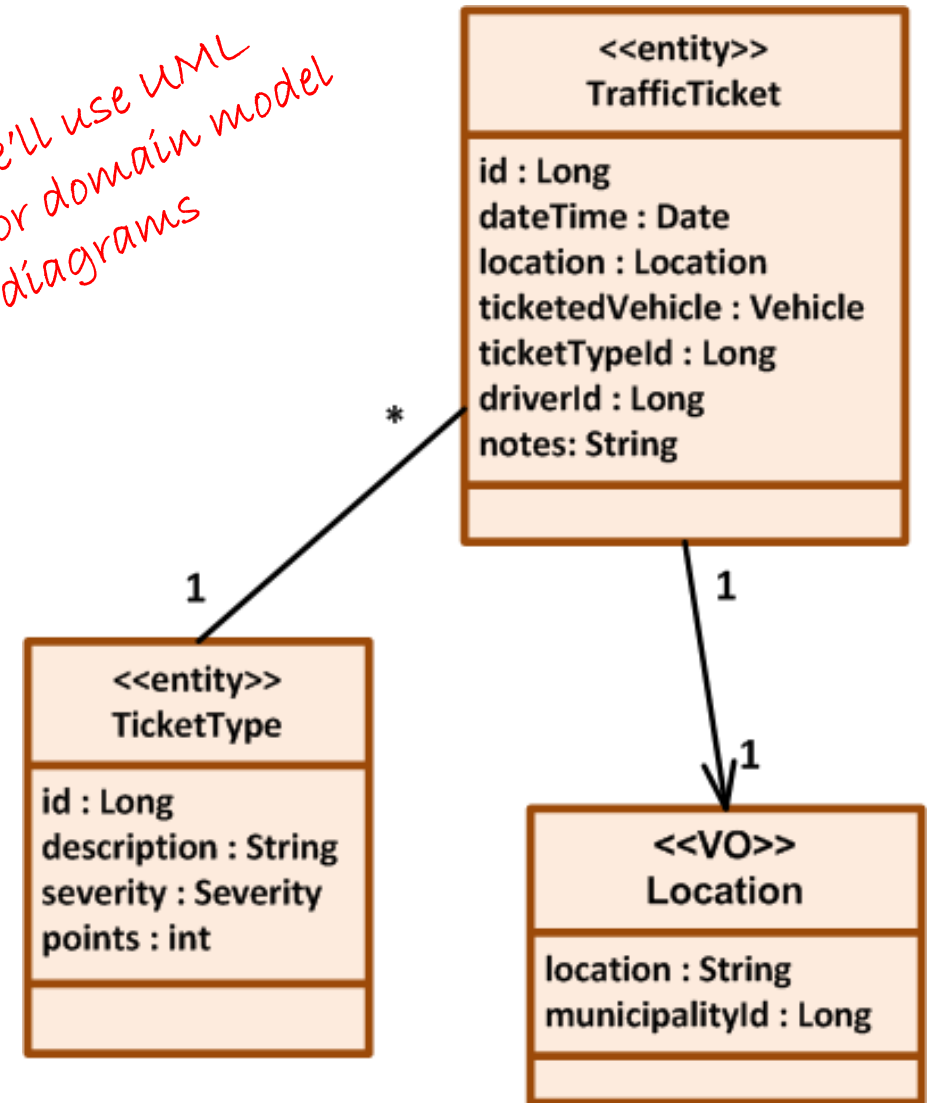
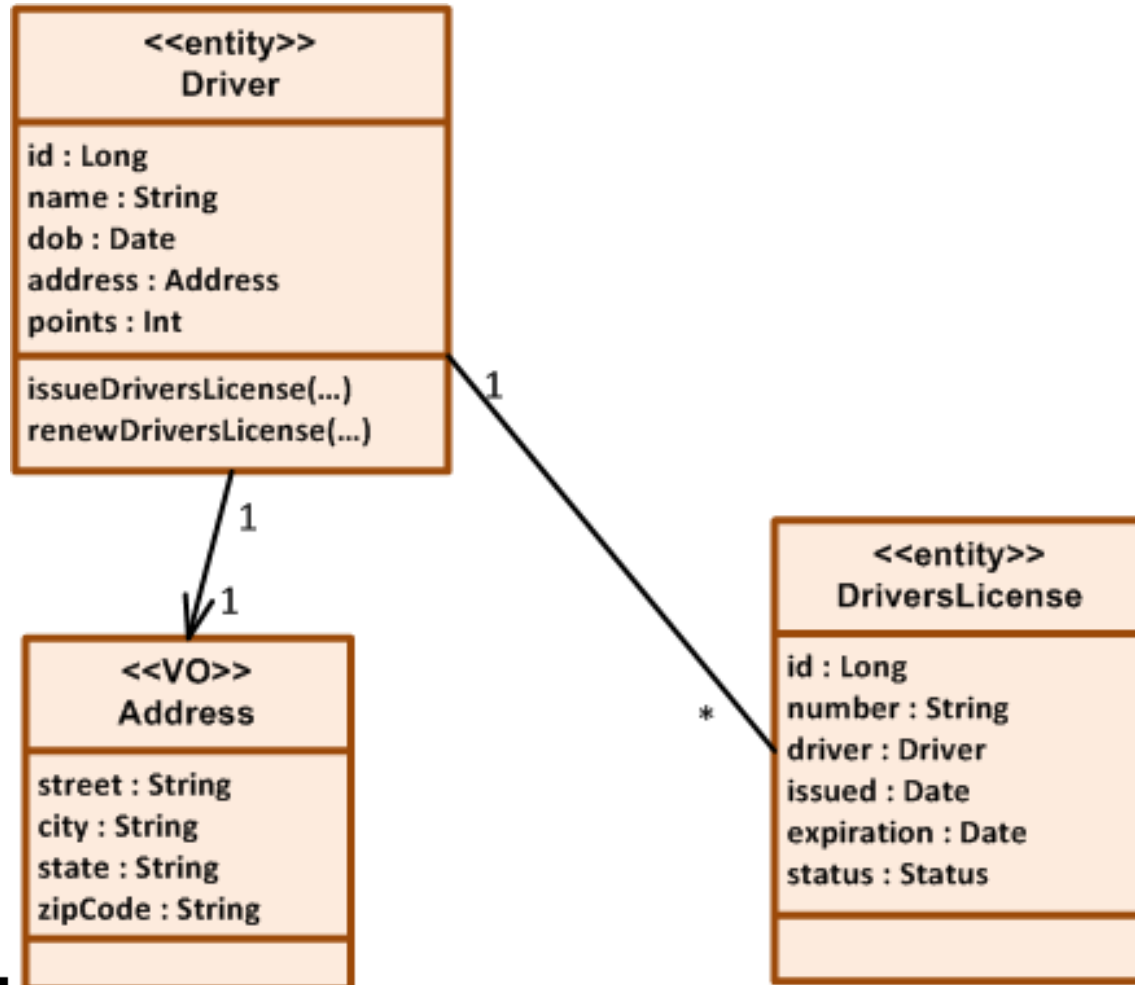
Value Object

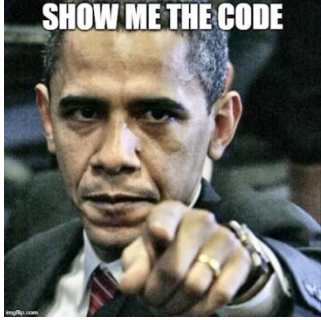
- Value objects represent characteristics or values in an entity
- They model a concept by composing related attributes
- Examples:
 - Address, Amount, Distance, Price, Geolocation
- VOs should be immutable—if the entity value changes, replace the value object with a new value object
- If entity-value relationship is
 - one-to-one, VO fields can be persisted inline with entity
 - one-to-many, serialize VOs and persist inline, or create separate table with FK

VOs don't have ID or life cycle

Entity and VO examples

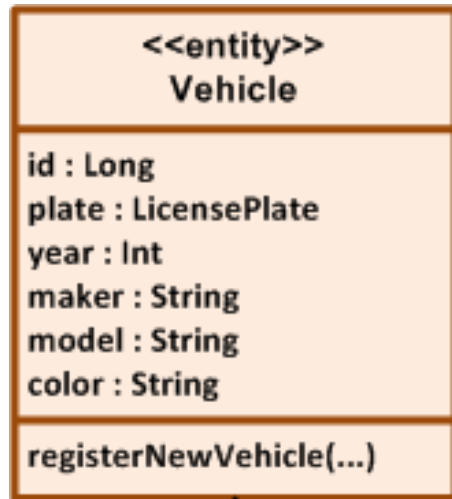
We'll use UML for domain model diagrams





One more example

Example of Entity using @Entity JPA annotation (ORM mapping details omitted)



```
@Entity
class Vehicle(
    @Id
    var id: Long,
    @Embedded
    var licensePlate: LicensePlateVO,
    var year: Long,
    var maker: String,
    var model: String,
    var color: String
)
```

Example of "inline" VO using @Embeddable JPA annotation

```
@Embeddable
class LicensePlateVO(
    var plate: String,
    var city: String,
    var state: String
)
```


Aggregate

- An aggregate represents a cohesive business concept, such as Employee, Loan, Purchase Order
- An aggregate has one or more entities and zero or more value objects
- An aggregate often has 1-2 entities, but aggregates with more entities are common, especially when there's a class hierarchy
- One entity is the **aggregate root**

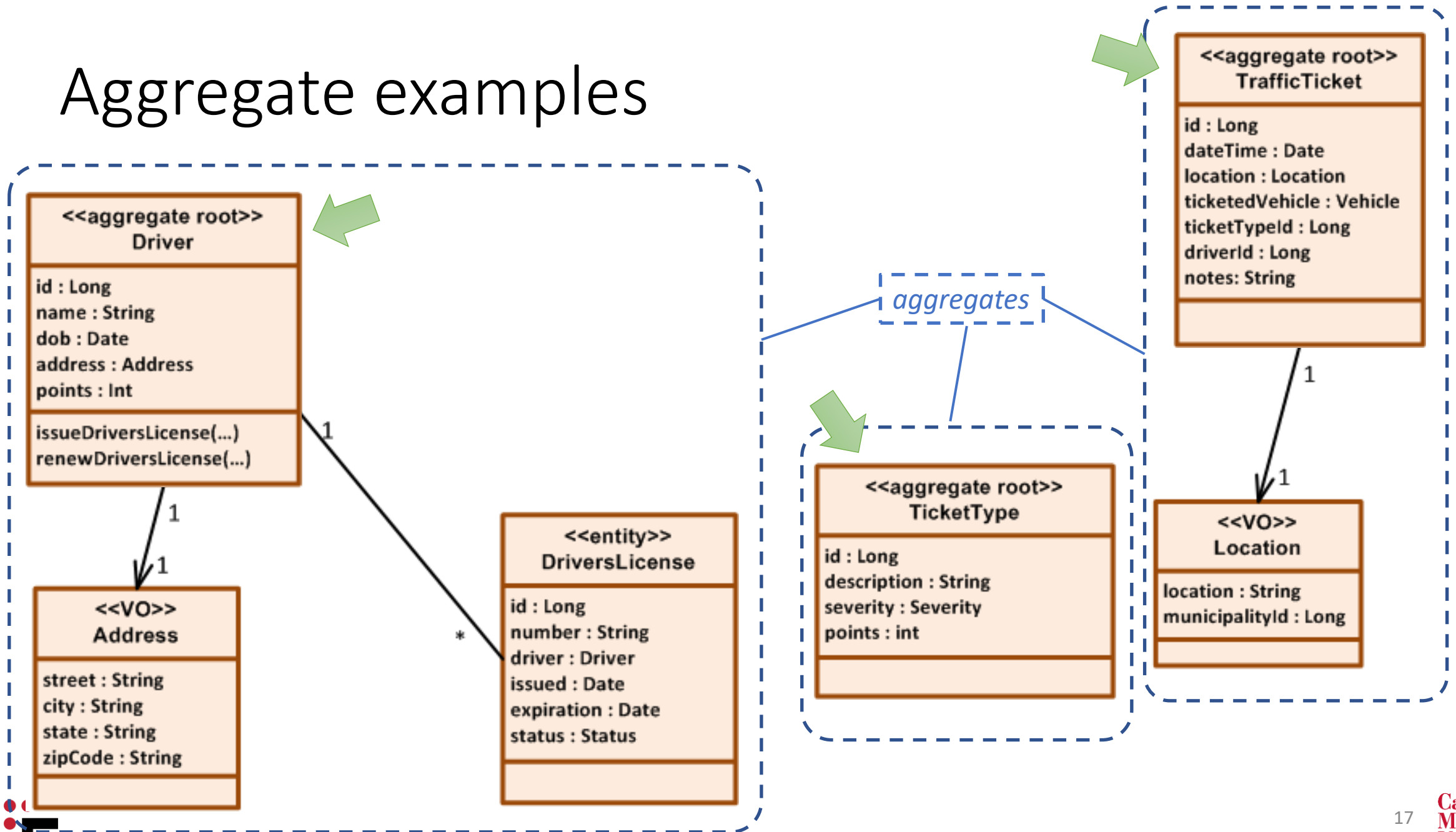
External objects/services only see the aggregate through the aggregate root

Aggregate transactional consistency

- An aggregate defines a (transactional) consistency boundary
- It remains transactionally consistent throughout its lifetime
- It is often loaded in its entirety from the database
- If an aggregate is deleted, all of its objects are deleted

A database transaction should touch only one aggregate

Aggregate examples



Inter-aggregate references

- Aggregate A may reference aggregate B
- The reference must use the ID of aggregate B

Traditional OO way 🤔

```
@Entity
class TrafficTicket(
    var id: Long,
    var dateTime: Date,
    var location: LocationVO,
    var vehicle: Vehicle,
    var ticketType: TicketType
) {
    var driver: Driver? = null
    var notes: String? = null
}
```

DDD way 😊

```
@Entity
class TrafficTicket(
    var id: Long,
    var dateTime: Date,
    var location: LocationVO,
    var vehicleId: Long,
    var ticketTypeId: Long
) {
    var driverId: Long? = null
    var notes: String? = null
}
```

Master, can VOs contain business logic?

Can a VO have a reference to an entity?

Can a reference to an entity be an object reference?

How do I choose the root of the aggregate?

What if my operation requires updating multiple aggregates

Yes. For example, it's common to add self-validation logic to VOs

Yes, it can

Yes if within an aggregate. Use the ID to reference aggregate roots outside

Inspect behavior not structure and you will see the aggregate root

Instead of transactional consistency you may only get eventual consistency



Repository



- The repository is an in-memory abstraction of all elements of a type
- The scope of a repository is an aggregate, so the repository represents the collection of elements in an aggregate
- It provides an interface around the aggregate root with functions to:
 - add and remove objects
 - query objects based on some criteria
- It hides the persistence technical details

```
interface TrafficTicketRepository : JpaRepository<TrafficTicket, Long> {  
    fun findByDriverId(driverId: Long) : List<TrafficTicket>  
}
```

*Example of repository using
Spring's JpaRepository interface*

*Spring "automagically" creates impl class
for you with several CRUD functions
(some seen in this test class)*

```
@RunWith(SpringRunner::class)  
class TrafficTicketRepositoryTest {  
  
    @Autowired  
    lateinit var repository: TrafficTicketRepository  
  
    @Test  
    fun `insert and retrieve a traffic ticket`() {  
        var ticketA = TrafficTicket(1, Date(), LocationVO("Hot Metal Bridge", 123), 1234, speedingTicket)  
        ticketA.driverId = 999  
        repository.save(ticketA)  
        var ticketB = TrafficTicket(2, Date(), LocationVO("Times Square", 321), 4321, parkingTicket)  
        ticketB.driverId = 111  
        repository.save(ticketB)  
        var aTicket = repository.findByDriverId(999)[0]  
        assertEquals(aTicket.id, ticketA.id)  
        repository.deleteById(2)  
        assertEquals(1, repository.count())  
    }  
}
```


Domain Service

- Domain services contain business logic in the domain that doesn't fit a specific entity or VO
 - Logic that requires access to 2+ entities, possibly from different aggregates
 - Logic that does not use data from any entity
- They are stateless and have no id
- They may rely on infrastructure elements, in which case
 - the domain service *interface* is in the domain layer
 - the domain service *implementation* is in the infrastructure layer

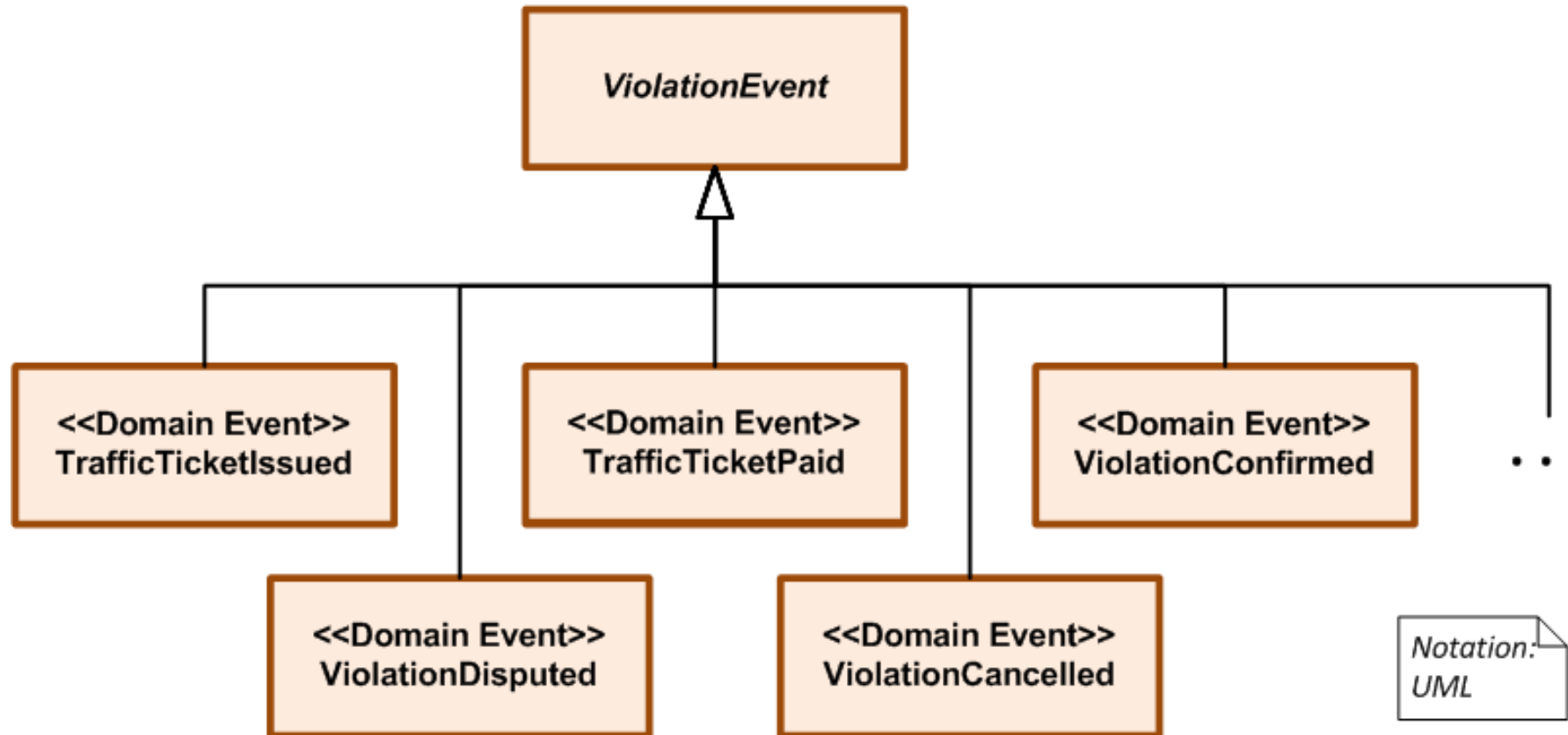
Entity-specific business logic remains in the entities

Domain events

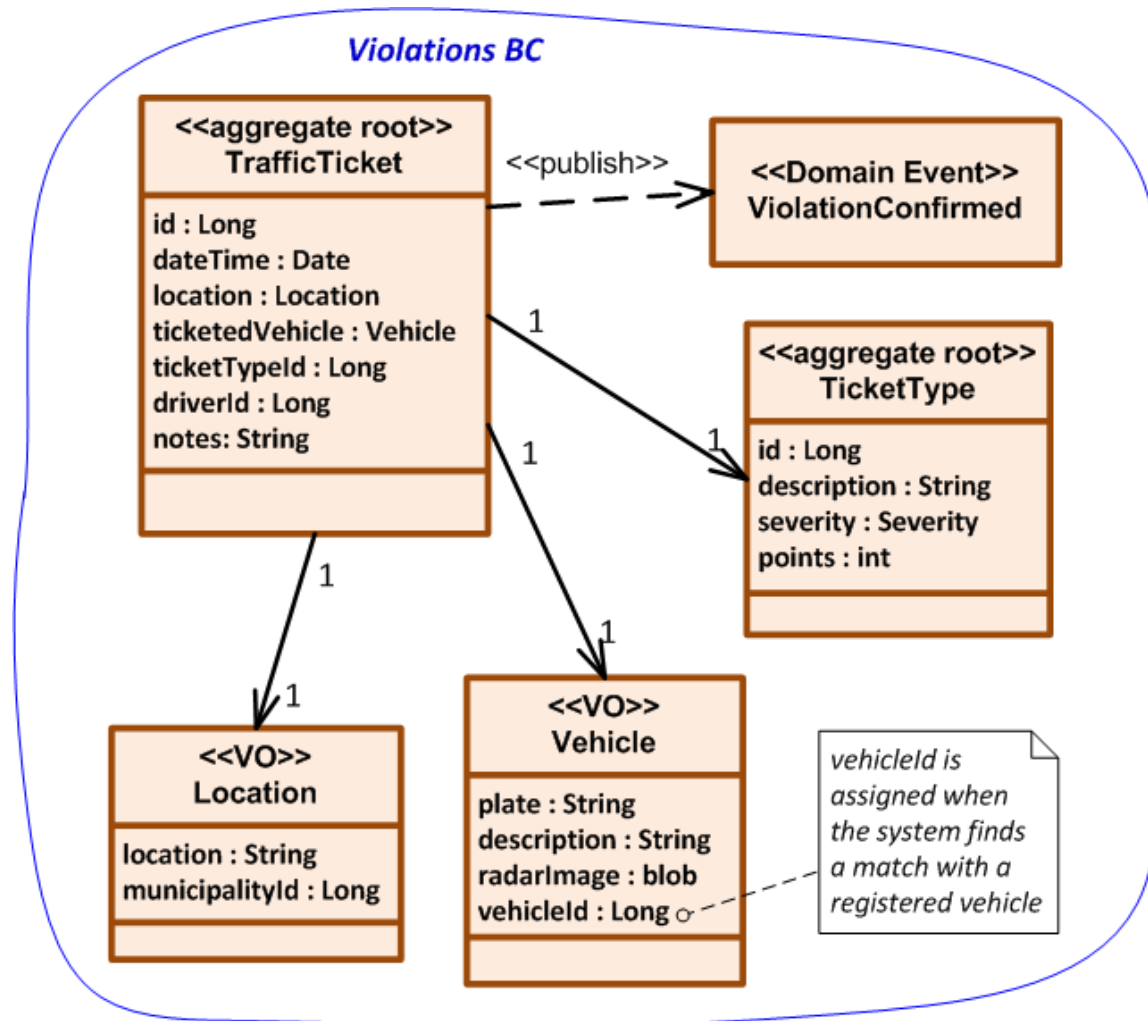
- A domain event
 - is something of interest that has happened to an aggregate
 - should be expressed in past tense
 - typically represents state change
 - should be represented by a class in the domain model
 - may be organized in an event hierarchy
- Examples:
 - OrderShipped
 - TimeOffScheduled
 - ReservationCanceled
 - UndoReservationCanceled

A domain event is immutable and cannot be rejected, though it can be undone by a corresponding undo/correction event

Domain event examples



Domain events are objects in the model



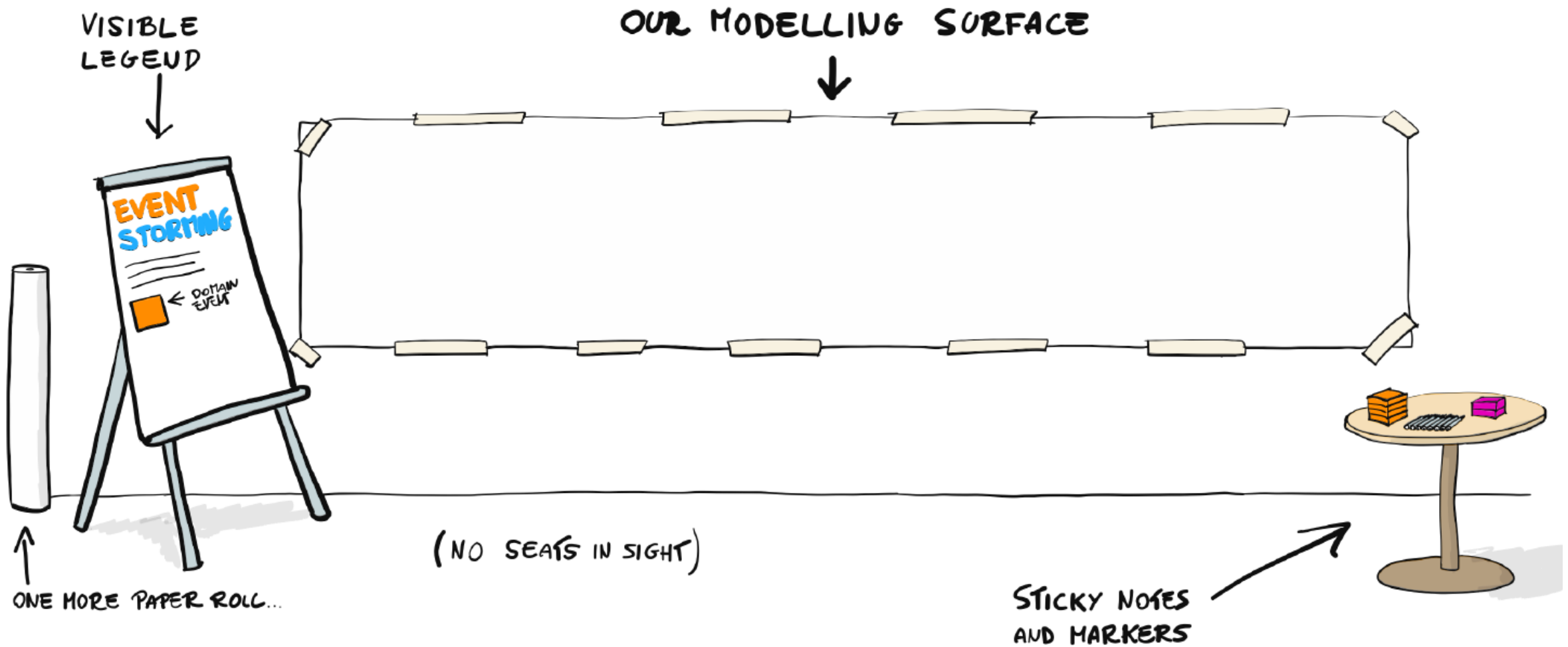
Event Storming

Learning Event
Storming is out
of scope for
this course

- Workshop for exploring domains and eliciting events in a collaborative way



Source: [Paul Reyner's talk at SATURN 2017](#)

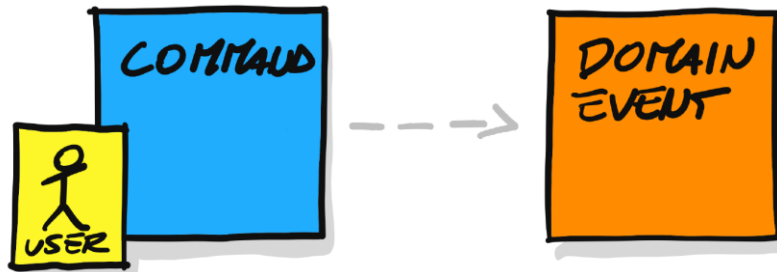


Source: Alberto Brandolini's book (leanpub.com/introducing_eventstorming)

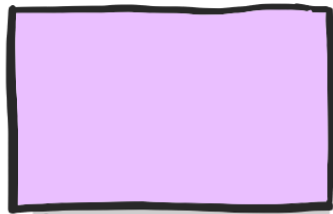


THIS IS A **DOMAIN EVENT**

- **ORANGE** STICKY NOTE
- VERB AT **PAST TENSE**
- **RELEVANT** FOR DOMAIN EXPERTS



USER INITIATED ACTION



POLICY

THIS IS OUR **POLICY**
SOMETIMES IT'S AUTOMATED,
SOMETIMES PEOPLE JUST
HAVE TO REMEMBER

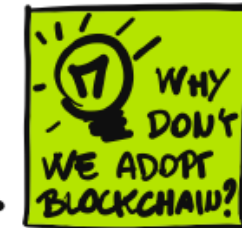
A PERSON
PLAYING A ROLE IN
OUR MODEL



A SYSTEM PLAYING
A ROLE IN OUR MODEL
(NO FEELINGS INVOLVED)



THIS IS A HOTSPOT: GOES
FOR PROBLEMS, RISKS, ETC.

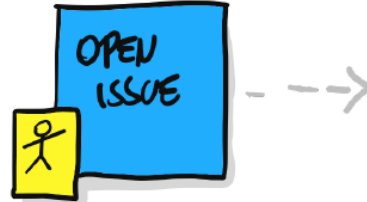


THIS IS AN OPPORTUNITY:
IT WILL CAPTURE IDEAS

Source: Alberto Brandolini's
book (leanpub.com/introducing_eventstorming)

WHERE ARE DOMAIN EVENTS COMING FROM?

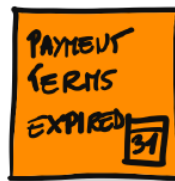
MAYBE AN ACTION
STARTED BY A USER



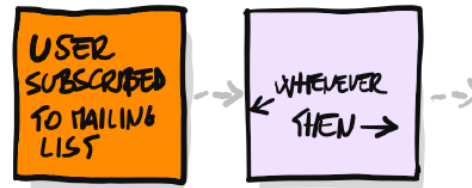
MAYBE THEY'RE COMING
FROM AN EXTERNAL SYSTEM



MAYBE THEY'RE JUST THE
RESULT OF TIME PASSING



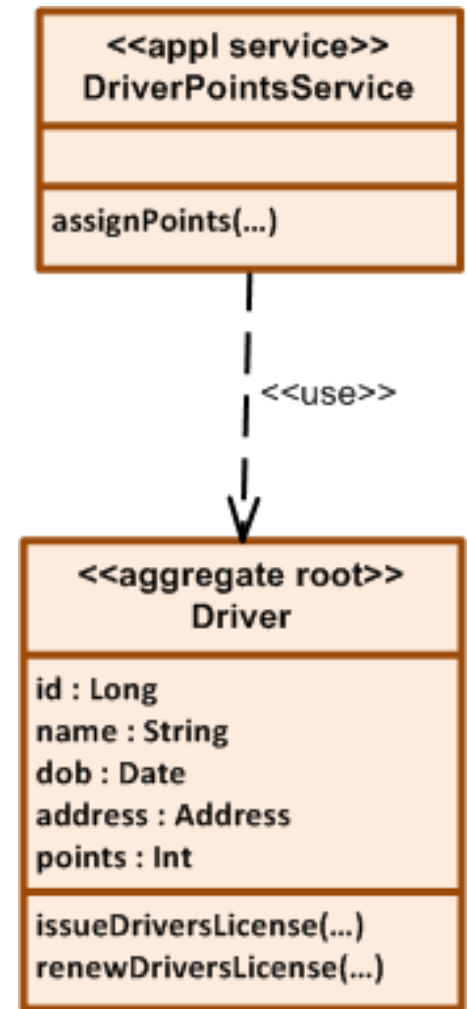
OR MAYBE, THEY'RE JUST
THE CONSEQUENCE
OF ANOTHER DOMAIN EVENT



Source: Alberto Brandolini's
book (leanpub.com/introducing_eventstorming)

Application Service

- Clients of the application should not have direct access to domain model elements (e.g., entities)
- The interaction should be through application services
- An application service
 - exposes the functionality to the clients of the application
 - offers operations that correspond to use cases
 - coordinates calls to domain elements
 - may access different aggregates
 - does not contain domain-specific business logic
 - deals with transactions, access control, logging
 - may call other applications





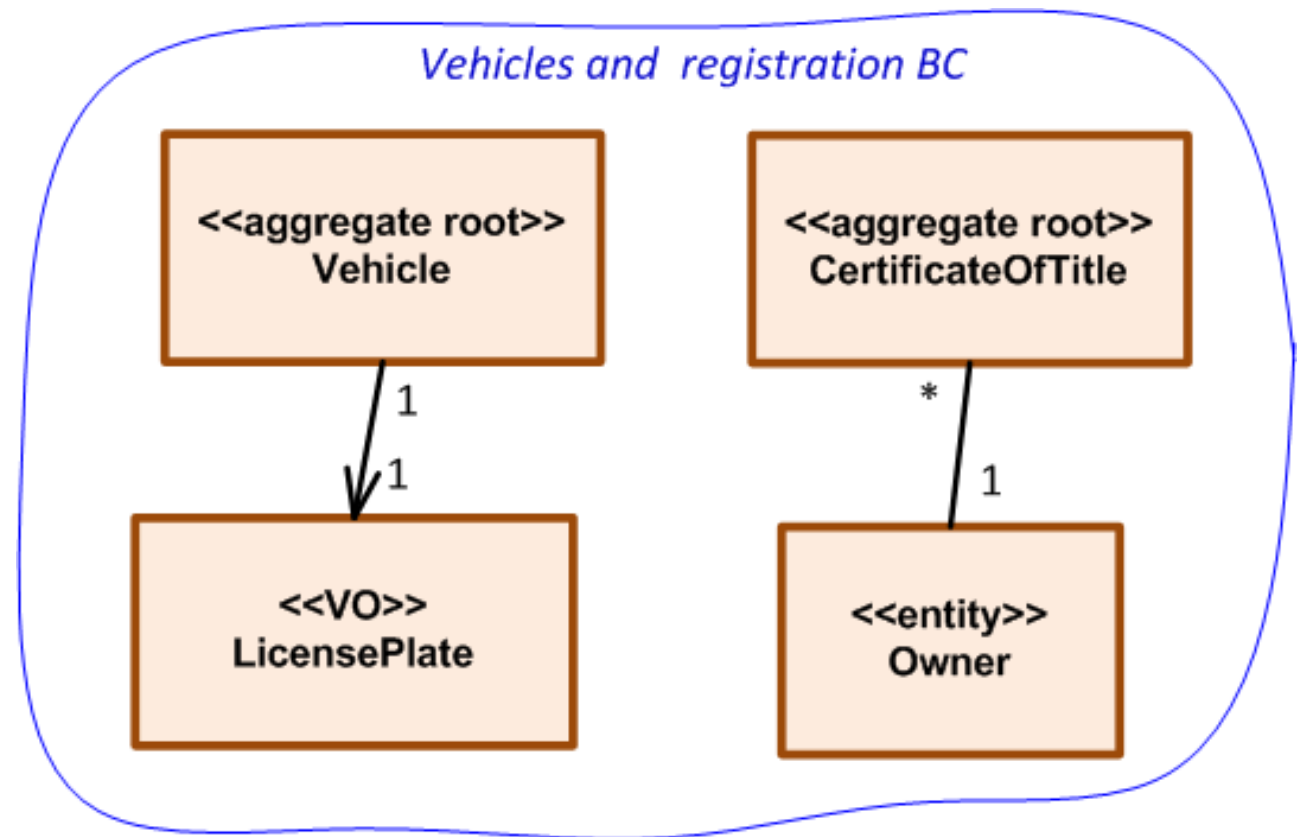
*Entity, VO, Aggregate, Repository, Domain Event, Domain Service, Application Service are **tactical modeling** elements*

*DDD is also concerned with the big picture and has **strategic modeling** elements*



Bounded Context

- A bounded context (BC) delimits the scope of a domain model
- It contains 1+ aggregates



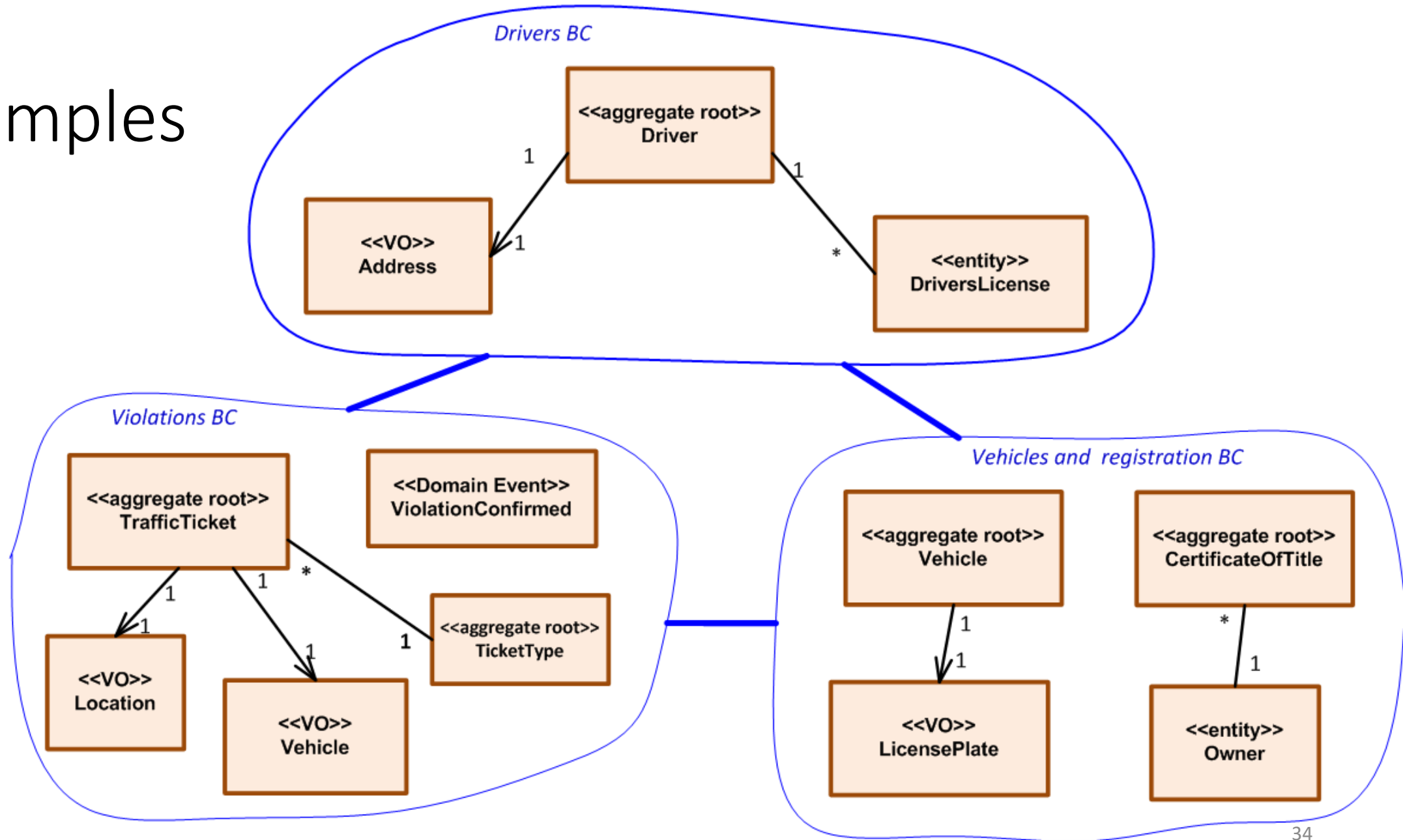
Scope of a Bounded Context

- The scope of a BC can be:
 - The entire domain model of a subdomain (recommended)
 - Domain models of 2+ subdomains (often happens with legacy or monolithic systems)
 - Part of the domain model of a subdomain (when the other part of the subdomain is implemented somewhere else)
- In practice...
 - The scope of a BC is often the scope of a traditional application system
 - BCs should not see each other's database directly
 - BCs are autonomous and a developer should be able to tell whether a concept is in or out of a BC

But a concept (e.g., customer) may exist in different BCs with different attributes

No FKs between BCs

BC examples

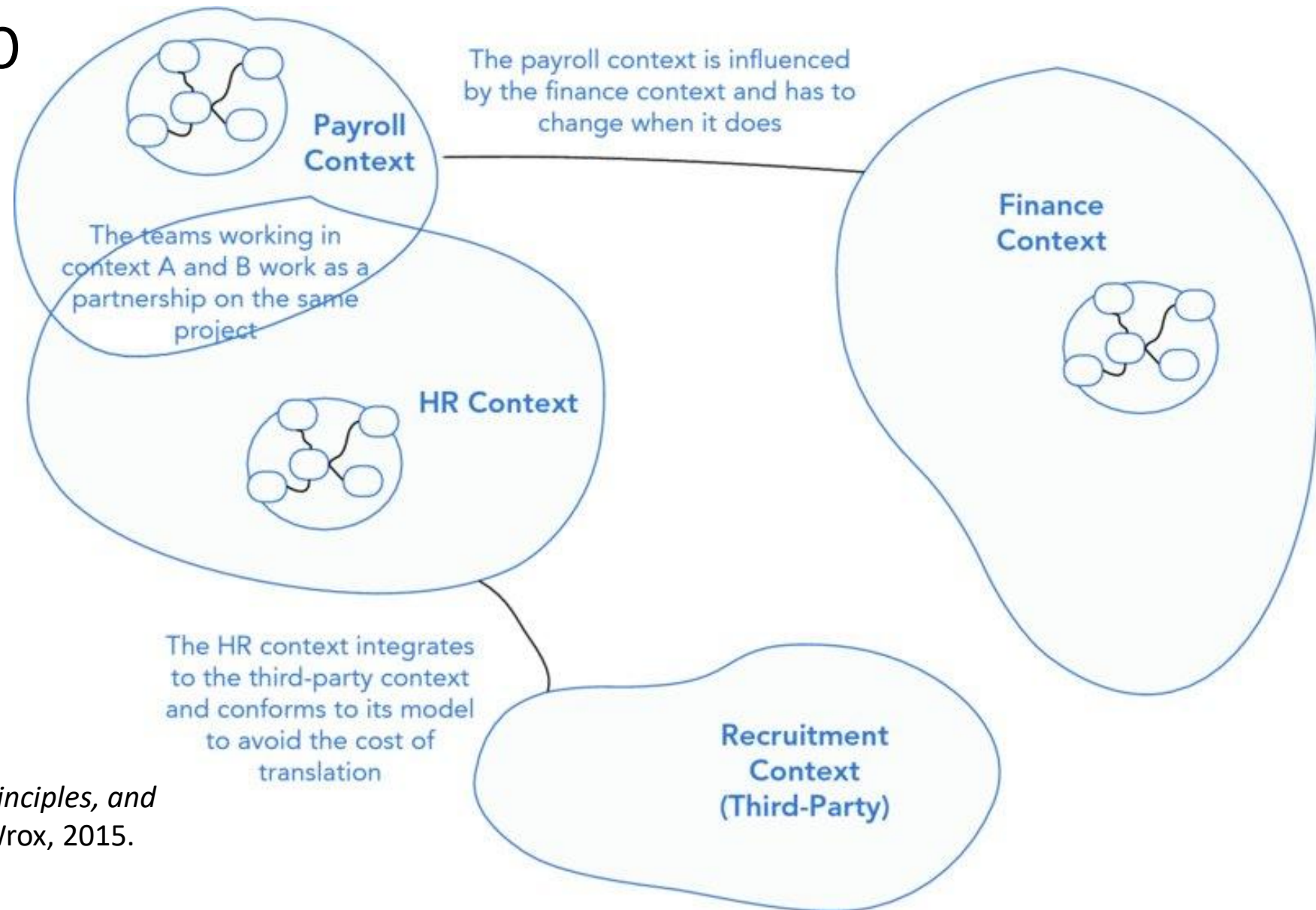


Thick blue line
indicates BCs
interact

Context Map

- A simple diagram that shows how BCs interact
 - The context map shows how *our* BC will interact with the others
 - Usually the other BCs (applications) already exist
- BC A (downstream BC) may use services of BC B (upstream BC)
- The relationships in a context map involve
 - technical aspects of the design
 - non-technical aspects, such as organizational forces and the maintenance status of each BC

Context map example



From: Millett, S; Tune N. *Patterns, Principles, and Practices of Domain-Driven Design*. Wrox, 2015.

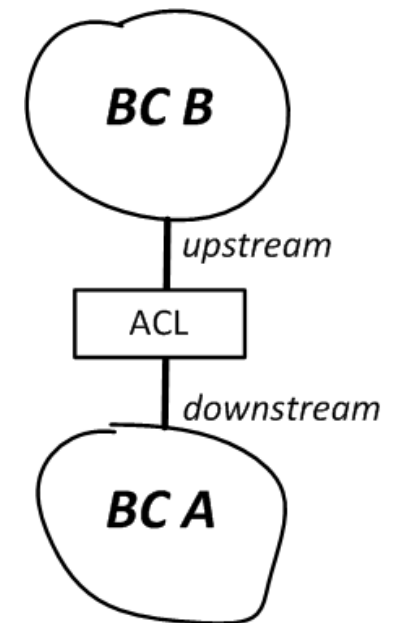
Context map relationships

- There are different patterns for context map relationships:
 - Anticorruption Layer
 - Shared Kernel
 - Separate Ways
 - Partnership
 - Customer-Supplier
 - Conformist
 - Open Host Service + Published Language

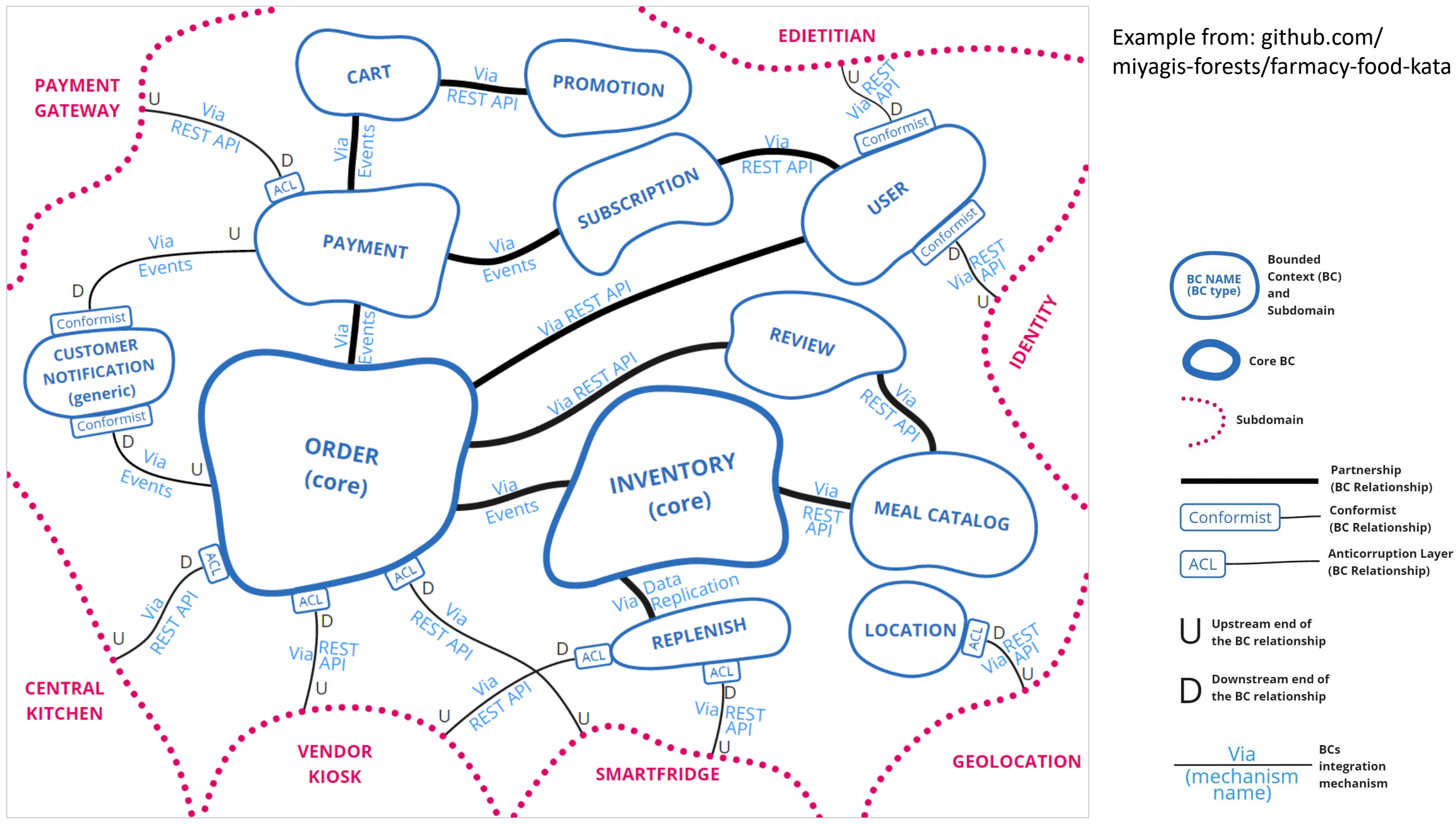
Anticorruption Layer (ACL)

- Problem
 - BC A needs to interact with BC B
 - Their domain models are different
 - “Adapting” BC A to BC B’s interfaces can lead to “corruption” of BC A’s model
- Solution
 - Create an isolation layer that offers an interface that adheres to BC A’s model
 - This isolation layer adapts and translates to BC B’s interfaces
 - It contains translation logic only, not business logic
 - It promotes loose-coupling across BCs

Technically, the ACL can take many forms



Example from: github.com/miyagis-forests/farmacy-food-kata



BC relationships – things to remember

Mind the organizational relationships between teams

BCs should not see each other's database directly

There should be no FKs between BCs

A concept (e.g., Customer) may exist in different BCs with different attributes

Each BC establishes its own ubiquitous language

Ubiquitous Language

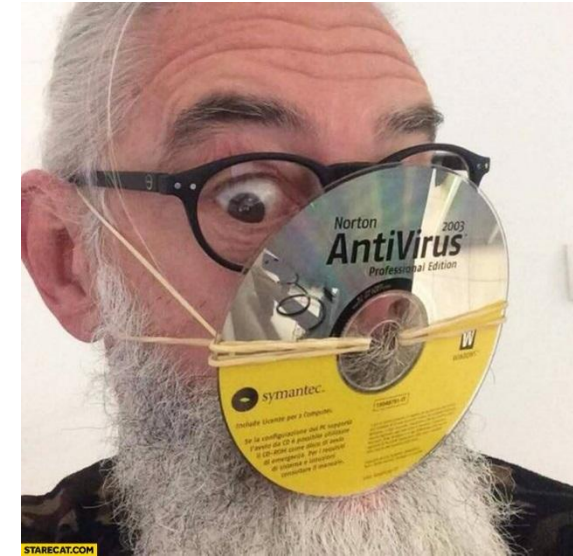
- Language shared by domain experts and devs on the same team
- Captures concepts, terms, jargon of a specific business domain
 - But it's not simply the lingo used by domain experts
 - It's influenced by that lingo but it's the language that consolidates for the BC as domain experts and devs discuss the domain model
- Should be used in the domain model *and* the code

There is one ubiquitous language per bounded context

Ubiquitous language (in context)

```
result = prime * result + GenomeSequenceNumber.hashCode();  
result = prime * result + ((Host == null) ? 0 : Host.hashCode());  
result = prime * result + ((SubType == null) ? 0 : SubType.hashCode());  
result = prime * result + ((VirusName == null) ? 0 : VirusName.hashCode());
```

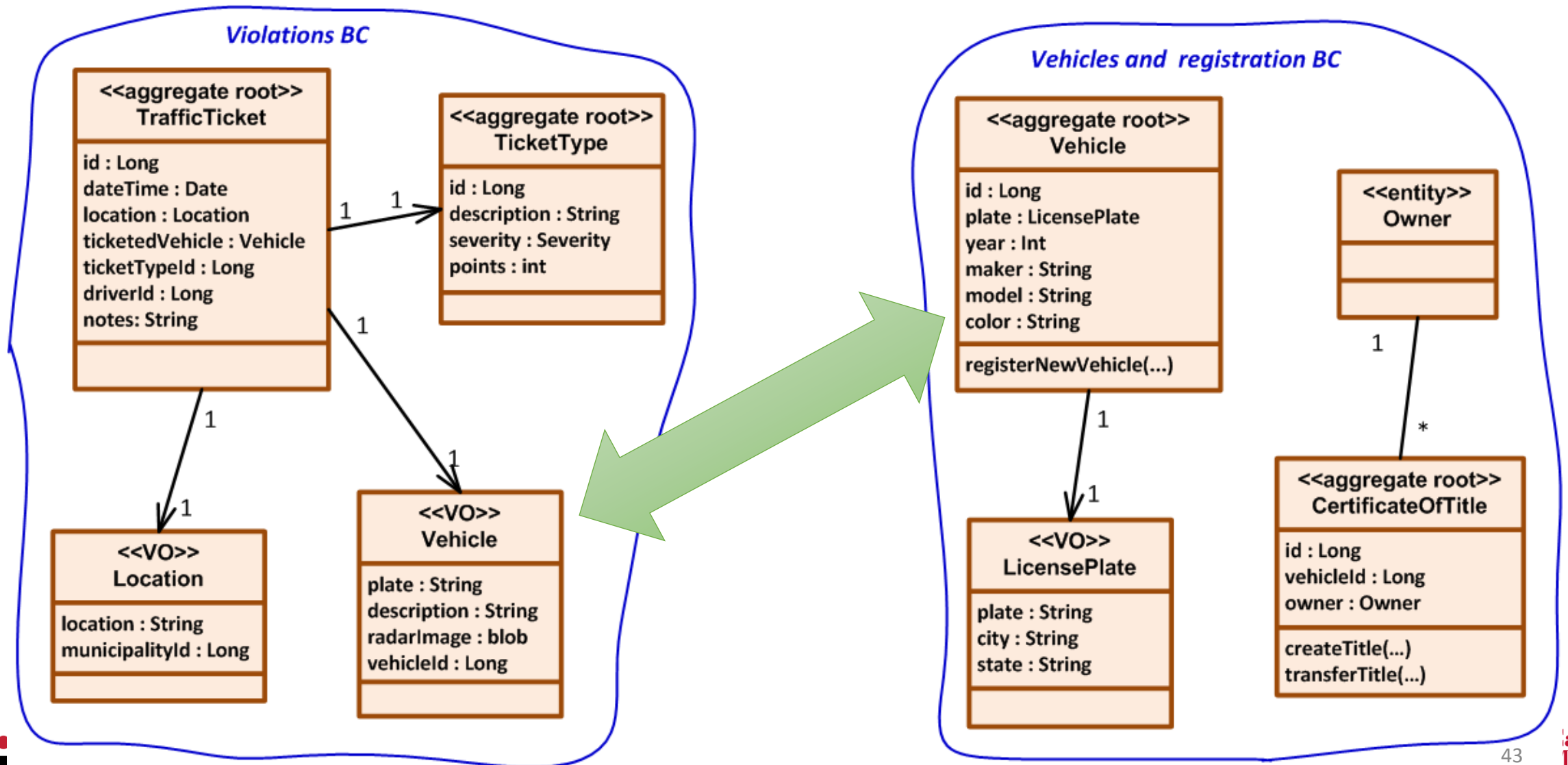
```
VirusTotal virus_total;  
std::cout << "\n\nScan Scheduled files [y/n]: ";  
std::getline(std::cin, get_option);  
if (get_option == "y") obj.scan_scheduled_files();  
obj.display_suspicious_files();
```

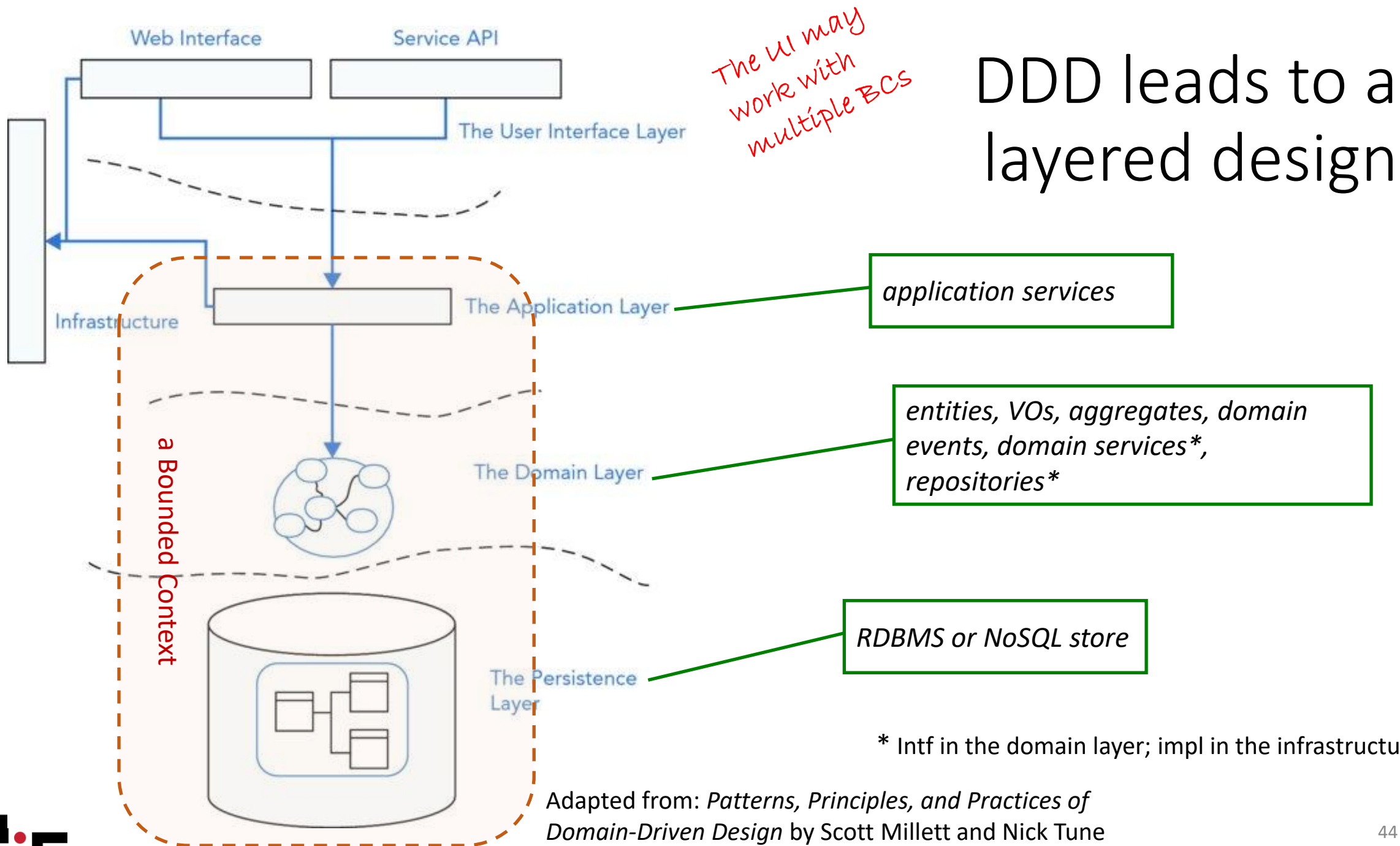


- The same term may mean different things in different BCs
- A domain object may show in the model of different BCs with different attributes

Context is king!

A term in different contexts – an example





*GenAI struggles
with domain
modeling...*

... because:

- *domain modeling requires deep, iterative conversations with domain experts to unearth unspoken assumptions*
- *the LLM is likely not trained on the domain events and ubiquitous language of the target business domain*
- *modeling BC interactions requires knowledge beyond the technical aspects (e.g., relations between teams)*
- *mapping the domain may overload the GenAI context window leading to hallucinations and inconsistencies*





Questions? now or later

Paulo Merson
pmerson@cmu.edu